

SECOND EDITION

THE

PS2

LESSONS
LEARNED
(SO FAR)

PROGRAMMING
LANGUAGE

NINJA DYNAMICS
JUNE 2026

Contents

Chapter 1 - The PS2 Mental Model

Chapter 2 - Source Boundaries and Port Architecture

Chapter 3 - Boot, IOP Bring-Up, and Standalone Reality

Chapter 4 - Toolchain and Build-System Traps

Chapter 5 - GS Display, Framebuffers, and Video Modes

Chapter 6 - Aspect Ratio and Full-Frame Coverage

Chapter 7 - ps2gl State, Batching, and Frame Boundaries

Chapter 8 - Depth, Blending, and 2D Primitives

Chapter 9 - Geometry, Clipping, Projection, and Display Lists

Chapter 10 - Texture Formats, Alpha, and Filtering

Chapter 11 - Manual Mipmaps, Dither, and GS Texture Precision

Chapter 12 - GS and EE Memory Management

Chapter 13 - Effects and Scene-Specific Rendering

Chapter 14 - Input, Debug UI, and Save Data

Chapter 15 - SPU2 SFX Fundamentals

Chapter 16 - Music Streaming and Custom IOP Modules

Chapter 17 - Validation Doctrine

Chapter 18 - ps2gl and ps2stuff Fork Delta

Appendix A - Quick Rules

Appendix B - Case Study: HyperSolar PS2 Port in One Page

The PS2 Programming Language

Lessons Learned (So Far)

by Ninja Dynamics

Public Release Edition, June 2026

This handbook is distilled from HyperSolar's PS2 work. It is written for the person who already has the SDK, already has the manuals, and is now staring at a black screen, a silent SPU2, a wrong-colored CLUT, or a render path that works in PCSX2 and fails on hardware.

It is not a replacement for Sony's manuals. It is the scar tissue layer: the rules that only became obvious after ps2gl, the GS, SPU2, SIF RPC, PCSX2, ps2link, and real hardware disagreed with one another.

The promise is practical: every chapter should leave you with a rule you can use in a real game, a failure mode you can recognize, and a reference you can open when you need to go deeper.

Public release note: this edition is written to stand alone. Implementation names, helper names, and subsystem names appear as concrete examples, not as dependencies the reader must have.

Why does this exist

Most of what is hard about the PS2 is not in any one manual. The Sony manuals are excellent first-principles references — they tell you exactly what every GS register does and exactly how an SPU2 voice is structured. What they cannot tell you is which of those facts will quietly ruin your week: that a 16-bit color buffer paired with a 24-bit Z buffer renders perfectly in an emulator and corrupts on hardware; that `pglFinish()` destroys your GL context instead of flushing it; that a buffer-loaded IRX can report success and never register its RPC server. Those are emergent properties of the *combination* — EE plus GS plus IOP plus SPU2 plus a forked OpenGL-ish library plus an emulator plus a network loader — and they only show up when the pieces disagree.

This handbook is the record of those disagreements and how they were resolved. It is deliberately opinionated: where a rule has survived real hardware validation it is stated plainly, and where a fix is empirical-but-unproven it says so.

How to read this handbook

Each chapter is self-contained and built around the same spine: a rule you can apply, the failure mode that makes you reach for it, and a pointer to the Sony manual chapter that explains the underlying hardware. Read straight through for a tour of the whole port, or jump to the chapter that matches your current black screen. Chapter 1 is the mental model everything else assumes; Appendix A is the whole book compressed to a checklist; Appendix B is the port told as a timeline.

Conventions

- **"Validate on hardware"** appears often and is not boilerplate. PCSX2 is a different witness from a real console (see Chapter 17). When a rule is hardware-only, the text says which class of

bug the emulator hides.

- **Manual citations** name the Sony manual and the relevant chapter so you know where to open for first-principles detail. The GS User's Manual splits into *Local Memory* (Ch 2, pixel-storage modes and page geometry), *Drawing Function* (Ch 3, texture mapping, alpha-blending, dithering, pixel test), *Image Data Transmission* (Ch 4, EE↔GS transfers and GIF), *CRTC* (Ch 5, display/video output), and *Registers* (Ch 7) — those splits are referenced by number throughout.
- **Confidence is marked.** Proven rules are stated as rules; inferred ones are flagged as empirical so you do not inherit a guess as a fact.

Sources and reference shelf:

- HyperSolar PS2 development notes, distilled and rewritten for public release.
- Ninja Dynamics, ps2gl and ps2stuff Fork Delta Report, June 2026.
- Sony PlayStation 2 GS User's Manual: GS registers, display environment, pixel storage modes, CLUTs, texture registers, alpha blending, scissor, GIF packet formats, and image transfers.
- Sony PlayStation 2 GS User's Manual Supplement: GS errata and corner cases.
- Sony PlayStation 2 EE Overview Manual: EE, VU0/VU1, GIF, VIF, DMAC, IPU, and the system architecture.
- Sony PlayStation 2 EE User's Manual: DMAC, GIF, VIF, timers, INTC, scratchpad, and transfer behavior.
- Sony PlayStation 2 EE Core User's Manual: R5900 CPU, caches, MMU, COP0, and pipeline behavior.
- Sony PlayStation 2 EE Core Instruction Set Manual: R5900 and MMI instructions.
- Sony PlayStation 2 VU User's Manual and VU Instruction Manual: VU architecture, micro-mode, and microinstructions.
- Sony PlayStation 2 SPU2 Overview Manual: two cores, 24 ADPCM voices per core, 2 MB SPU2 RAM, voice registers, PITCH, ADSR, KON/KOFF, VMIX, and mixing.
- Community PS2 texture-format references are useful quick companions to the GS manual's PSM and CLUT tables.

Manual citations in this handbook name the relevant Sony manual by title. No quoted manual text is required to use this handbook; the manual references are there so you know which hardware chapter to open when you want first-principles detail.

Chapter 1 - The PS2 Mental Model

Relevant manuals: EE Overview Manual Ch 2 (Architecture Overview) and Ch 3 (Functional Overview) for how the EE core, VU0/VU1, GIF, VIF, DMAC, and IPU connect and feed the GS; GS User's Manual Ch 1 (Overview) for the GS block configuration; SPU2 Overview Manual Ch 1 (Overview of SPU2) for the audio side.

Before any specific rule, internalize the shape of the machine — almost every trap later in the handbook is really a consequence of getting one of these seams wrong (an EE pointer handed to the IOP, a stream forced through a DMA boundary, an emulator standing in for a CPU it does not fully model).

The PS2 is several machines tied together by DMA and RPC-like boundaries. Treat those boundaries as real architectural seams.

The EE runs the game, simulation, asset orchestration, and most rendering glue. The VUs and ps2gl handle transform/render submission for supported paths, but the case-study renderer also uses an EE CPU software clipper for geometry that ps2gl cannot safely clip. The GS is not a modern forgiving GPU; it is a rasterizer with strict VRAM page geometry, fixed-point coordinate limits, explicit texture registers, and very sticky state.

The IOP owns many services that look "system level" from the EE: modules, filesystems, memory cards, libsd/ps2snd, USB mass storage, and custom IRX code. The EE speaks to it through SIF RPC and DMA. If an API takes a pointer and the work happens on the IOP, ask which CPU's address space the pointer belongs to.

The SPU2 is a local-memory ADPCM synth, not a stream-player. It gives excellent per-voice pitch/volume control once samples are resident in SPU2 RAM. Long music requires streaming into SPU2 memory while the voice plays.

PC, PCSX2, ps2link, and real hardware are four different witnesses. PCSX2 is great for correctness and crashes, but it can hide GS display-register bugs, texture precision artifacts, mip blend quantization, and some hardware-only filtering behavior. ps2link can mask standalone IOP boot problems because it leaves the IOP in a friendlier state than a disc, USB boot, or emulator launch. Real hardware remains the final judge.

Chapter 2 - Source Boundaries and Port Architecture

Relevant manuals: mostly architectural — EE Overview Manual Ch 1 (Architecture Policy) frames why the EE/GS/IOP division of labor exists in the first place. This chapter is otherwise engineering discipline drawn from the case study, not hardware behavior.

This chapter is about a different kind of correctness than the rest of the handbook: not "what does the hardware do" but "where does each responsibility live in your code." It earns its place because every PS2-specific bug is cheaper to fix when the platform code is a thin, parameter-fed library than when it has quietly grown into a second copy of the game with its own drifting scene order.

The PS2 backend should be a library, not a parallel game. In the case-study codebase that backend is named `playstation2.c`; in another project it may have another name. The important shape is a hardware/service layer that shared game and render code call into.

It may own:

- pad adapter code;
- embedded asset registry callbacks;
- GS-safe texture helpers;
- alpha conversion and blend-state helpers;
- the software clipper;
- display-list compile/call helpers;
- PS2 draw primitives such as skybox, floor, haze, particles, reticles, sun, speedlines, and lens flare;
- scene resource installers that turn already-loaded assets into PS2 helper meshes, display lists, or baked textures;
- memory-card blob I/O and low-level PS2 service code.

It should not:

- read `game.*`, except for the explicit pad-write adapter pattern;
- name game asset files directly;
- compose scenes;
- own stage sequencing, gameplay state transitions, timers, scoring, or sim behavior.

Every value derived from game state crosses into the PS2 helper as a parameter: camera, anchor, floor type, scroll values, model transforms, lighting, tint, bounding sphere, sun position, or effect intensity. This keeps the cross-platform files as the source of truth.

The scene graph should remain shared. In the case study, duplicated PS2 scene code drifted in draw order. That drift was not cosmetic: haze drew after particles and made explosions look behind fog. Keep new scene elements in the shared scene function and put PS2-specific behavior at leaf helpers.

When porting a cross-platform render function, grep the original for `game.stage->` and `game.*` reads. Each read is either a parameter the PS2 dispatch must forward or a sign that the helper boundary is wrong. The lava stage exposed this: a PS2 floor helper hardcoded the water floor composition and the lava stage became double-layered and transparent.

Stage-shaped resources are now real per-stage resources on PS2. Skyboxes and floors load from the stage table, call the CSV texture pipeline, and install the PS2 form. Only the active scene's GS resources should be resident. Mipped floors need a deep unload because their locked mip levels and global MIPTBP ownership outlive a plain texture unload.

The backend API has two classes of calls:

- public one-shot draw calls and public `*_begin()` calls must fence their own GS state;
- hot-loop `*_add()` or `*_draw()` calls between begin/end should be lean and inherit state from the matching begin.

The F22 inside-out bug is the warning label. Reticles restored generic raylib state that re-enabled culling. If `ps2_render_player()` trusts prior state, the reversed GLB winding gets culled from the visible side.

Chapter 3 - Boot, IOP Bring-Up, and Standalone Reality

Relevant manuals: EE Overview Manual Ch 2 (Architecture Overview) for the EE/IOP/SIF relationship; EE User's Manual for the DMAC and SIF transfer paths the RPC layer rides on. For audio-side consequences, see SPU2 Overview Manual Ch 1.

The IOP is the part of the machine most likely to lie to you at boot. It will accept a module, hand you back a positive id, and quietly never finish bringing it up — and the one tool you would normally use to see this (`printf`) is gone the moment you leave the network loader. This chapter is the bring-up ritual and the breadcrumb techniques that make a silent IOP visible.

Do standalone IOP bring-up before loading buffer IRX modules. A module loaded with `SifExecModuleBuffer` can return a positive module id and still never register its RPC server if the IOP is in the wrong inherited state. `ps2link` masks this because its loader leaves the IOP usable; PCSX2 and real disc/USB standalone boots do not.

The reliable sequence is:

```
SifInitRpc(0);
SifIopReset("", 0);
while (!SifIopSync()) { }
SifInitRpc(0);
sbv_patch_enable_lmb();
sbv_patch_disable_prefix_check();
```

Link `-lpatches` and include the IOP control and SBV patch headers. This belongs at the top of pre-init, before memory-card modules, `libsd/ps2snd`, BDM, custom IRX modules, or any other buffer-loaded code.

`SifIopReset` drops `ps2link` console and `host: fileio`. That is acceptable only if the game no longer needs `host:` after boot. Once assets are embedded and USB or disc paths are explicit, this makes the dev run behave like the shipped standalone run, which is exactly what caught the silent-audio bug.

Do not equate "works through `ps2link` on hardware" with "works standalone." The audio stack proved the danger: `ps2snd.irx` worked over `ps2link`, but in standalone boot its `BINDID_PS2SND` RPC never came up until the IOP reset and SBV patches were added.

Make the reset build-conditional so it does not cost you the dev loop. `SifIopReset` reboots the whole IOP, and under network boot `ps2link`'s ethernet driver (SMAP) lives there — so the reset kills `ps2link`'s network. The EE program keeps running, but `ps2client reset` / `printf` die and you must physically power-cycle to reload `ps2link` before the next deploy (telltale: `ping` answers right after a hard reset, then goes unreachable the instant the ELF runs). The reset is only needed standalone — under `ps2link` the IOP is already primed for buffer-loaded IRX — so guard it with a build flag: `#ifndef PS2_PS2LINK` around the reset / sync / re-init, defined only for the `ps2client` deploy goal, keeping `SifInitRpc(0)` and the SBV patches on both paths. The standalone build stays self-contained; the network-boot dev build keeps `ps2link` alive so reset/printf survive and iteration needs no power-cycle.

When debugging standalone or PCSX2 boot hangs, `printf` is gone. `ps2sdk printf` routes to `ps2link`, and PCSX2's trace log has hardware trace categories, not a program-console stream. Use GS color breadcrumbs:

```
BeginDrawing();  
ClearBackground(SOME_COLOR);  
EndDrawing();  
ps2_frame_end();
```

The screen freezes on the last completed color. PCSX2 logs can still distinguish exceptions from spins. If there is no EE exception but the log shows timeout loop skipping, suspect a bind loop or wait.

`nopdelay()` is expensive. It is about one million nops, roughly 17 ms per call. Big SDK-style loops survive only because they break on the first success. If a failure path can run to completion, a `100000 * nopdelay()` wait is effectively a hang. Use small bounded counts.

`sceSifBindRpc(cd, sid, 0)` blocks until the IOP replies. An unregistered server does not reply. For probes, use `SIF_RPC_M_NOWAIT`, poll `cd->server`, and re-send the bind occasionally because an unregistered SID can be silently dropped before the server appears.

Chapter 4 - Toolchain and Build-System Traps

Relevant manuals: EE Core User's Manual and EE Core Instruction Set Manual cover the R5900 the toolchain targets, but most of this chapter is GCC/binutils/ps2sdk behavior, not Sony hardware behavior.

These are the bugs that never reach the hardware because they stop at the compiler or linker — yet they cost real days, because the error messages point nowhere near the cause (a predefined macro turning a type into a number, a native-Windows assembler rejecting a POSIX path, a missing inline definition surfacing as a link error on one platform only).

The DreamSDK PS2 assembler is a native Windows executable. When MSYS invokes `mips64r5900el-ps2-elf-as.exe`, `.incbin "/c/Users/..."` fails because that path is not a Win32 path. Generate mixed Windows paths for `.incbin`, for example via `cygpath -m`, with a fallback for non-MSYS environments.

The EE compiler predefines `mips` as a macro. Do not name variables, fields, or parameters `mips`. It preprocesses to `1`, creating baffling syntax errors such as `CMMTexture* 1[6]`. Use `levels`, `mip_list`, or another name.

`RAYMATH_IMPLEMENTATION` must exist in exactly one translation unit for PS2. The PS2 toolchain plus `raylib4ps2` can fail to inline functions that other platforms resolve elsewhere, leading to undefined references such as `MatrixMultiply`. Put the implementation define in exactly one PS2-only backend translation unit.

When using forked libraries, link by explicit path and include their headers ahead of toolchain headers. In a sibling-fork layout, build `ps2stuff` as a local archive, link that archive by path, and do not install it over the toolchain copy. `ps2gl` must also compile against the forked `ps2stuff` headers if it uses fork-only APIs such as GS memory info.

Canary banners are a cheap sanity check for fork linkage. Make them print once at startup from `raylib`, `ps2gl`, and `ps2stuff`. A build-script stamp only rewrites an existing string; it cannot insert a missing `printf`.

Chapter 5 - GS Display, Framebuffers, and Video Modes

Relevant manuals: GS User's Manual Ch 5 (CRTIC) for the DISPLAY/PMODE registers, magnification, and video output; Ch 2 (Local Memory) for page geometry and framebuffer storage; Ch 7 (Registers) for BGCOLOR and the privileged register addresses; GS User's Manual Supplement for errata.

This is the chapter where the emulator lies most. The GS display path — sync timing, the DISPLAY register's magnification, framebuffer page alignment, the color/Z pairing, BGCOLOR — is largely normalized away by PCSX2 and only tells the truth on a real console with a real TV. Treat every rule here as hardware-validated, because each one was caught only after PCSX2 said it was fine.

Do not call libgraph's `graph_set_mode` or `graph_set_screen` on top of ps2gl. libgraph and ps2gl both program the GS display read-circuit registers. Calling libgraph corrupts ps2gl's display environment and can crash even in the default mode. Use `SetGsCrt` for sync-generator changes and ps2gl/fork helpers for the display registers that ps2gl owns.

`SetGsCrt` is only half of a video-mode switch. It changes scan timing, not the GS `DISPLAY` register's position, magnification, and size. ps2gl re-sends its display environment each frame, so a mode switch must update ps2gl's live display environment too.

Use the corrected ps2sdk constant: `GS_MODE_DTV_480P == 0x50`. An older lesson recorded `0x32`; that is superseded by the later PS2 video-mode work.

The case-study progressive layout is 640x448p, not 480p. The reasons are pure GS layout math:

- 448 is exactly 7 rows of 64-line 16-bit GS pages.
- 480 is 7.5 rows and needs padding to 512 lines in memory.
- 448p can share the same visible `SCREEN_W/H` and texture slot map as 448i.
- A 640x448 16-bit framebuffer uses 70 pages, matching the stock 448i footprint.

GS framebuffers must be page-row aligned. A GS page is 64x32 for 32/24-bit formats and 64x64 for 16-bit formats. Do not size framebuffers by `width * height * bpp / 8192` unless the height is page aligned. For a 16-bit 640x480 buffer, 480 lines address into 7.5 page rows; the bottom-right portion spills into the next buffer unless the reservation is rounded to 512 lines.

Color and Z formats must share page geometry. Pair 16-bit color with 16-bit Z, and pair 32-bit color with 24-bit Z. A 16-bit color buffer with 24-bit Z may look fine in PCSX2 but corrupts on real GS hardware because color and Z pages advance on different row heights.

The cost of 16-bit Z is coarser depth. Thin double-sided geometry can z-fight. Fix that with culling, not by switching to 24-bit Z. The case-study F22 GLB has reversed winding relative to ps2gl's `GL_BACK` convention, so the player model needs `glCullFace(GL_FRONT)`.

For DTV progressive, `DISPLAY.magH` must match the faster scan clock. Interlaced NTSC/PAL used `magH = 4`; DTV progressive needs `magH = 2`. Leaving the old value makes the image twice as wide on hardware. PCSX2 can normalize this away, so validate display-register work on a real console.

Runtime GS layout switching is possible, but the safe ritual matters:

- `pglFinish()` is not a flush; it destroys the GL context. Drain with `pglWaitForVU1()` and `pglWaitForVSync()`.
- Remove and rebuild GS memory slots, including locked framebuffer slots.
- Re-add framebuffer and Z slots, bind areas, set draw buffers, call `SetGsCrt`, set display buffers, then set the ps2gl video mode.
- Most textures re-upload lazily after the slot wipe, but mipmapped textures bake GS addresses into MIPTBP. Re-create them after the switch.
- Black-bracket the transition so uninitialized buffers and slot rebuilds do not scan out as VRAM garbage.
- The remaining blink is GS scan-generator resync; it is hardware behavior.

The GS `BGCOLR` register controls the margins outside the active framebuffer. `ClearBackground` does not touch it. Soft reset does not reliably clear it. Write it as a 64-bit store to physical `0x120000E0`, not a `kseg1` alias. Re-zero it after `EndDrawing()` and after video-mode application. Also re-zero it before known long mid-frame stalls such as scene loads; otherwise a previously stomped color can show in the side bars for the duration of the stall.

Chapter 6 - Aspect Ratio and Full-Frame Coverage

Relevant manuals: GS User's Manual Ch 5 (CRTC) for the display environment — how the read circuit magnifies and positions the framebuffer onto the output raster, which is the root cause of the stretch this chapter corrects.

A point that surprises most newcomers: a plain 4:3 PS2 framebuffer is *not* geometrically neutral. The display hardware stretches the stored lines to fill the raster, so "render 640×448 and forget about it" ships a subtly squished world. This chapter is the correction math and — just as important — the separation between *aspect* (a projection scale) and *placement* (a HUD re-anchor), which are easy to conflate and painful to debug once you have.

The 640x448 PS2 framebuffer is not geometrically neutral on a 4:3 display. The display stretches 448 lines to the TV raster height, making the scene vertically stretched relative to DC and PC. A 640x448 renderer can correct this by applying a centered horizontal scale:

```
xscale = (float)NATIVE_H / (float)SCREEN_H; /* 480 / 448 */  
if (ps2_wide_squeeze) xscale *= PS2_WIDE_SQUEEZE_X;
```

This baseline 4:3 correction and the 16:9 anamorphic squeeze use the same mechanism. The 3D pass post-multiplies projection; the 2D pass applies a matching centered transform. World-anchored 2D overlays stay aligned only if both passes use the same scale.

Aspect correction and placement are separate jobs. The pass scale corrects geometry; HUD layout must re-anchor edge positions with the inverse scale (`hud_ar_x()` in the game). Centered elements need no horizontal correction.

Any pass that replaces matrices inherits both jobs. Attract overlay passes load their own projection/view, so they must re-apply the squeeze and re-anchor edge-based screen coordinates before adding half-size offsets.

"Full-screen" fills are not simply `(0,0,SCREEN_W,SCREEN_H)` inside the PS2 2D aspect pass. Horizontally, use the inverse AR span from `hud_ar_x(0)` to `hud_ar_x(SCREEN_W)`. Vertically, overscan past `SCREEN_H` because the GS fill rule can drop the exact boundary row. Let the GS scissor clip the extra pixels. The same horizontal span rule applies to tiled texture backdrops.

Chapter 7 - ps2gl State, Batching, and Frame Boundaries

Relevant manuals: GS User's Manual Ch 3 (Drawing Function) for the drawing environment and per-primitive state, and Ch 4 (Image Data Transmission) for the GIF packet formats a batch ultimately becomes; VU User's Manual for the VU1-side nature of ps2gl's transform/submit paths.

The single most useful performance instinct on this hardware: stop counting vertices and start counting state changes. ps2gl turns a render-state transition or texture bind into a re-sent GS environment packet, which dwarfs the cost of the handful of vertices it surrounds. This chapter is the batching discipline that follows from that, plus the frame-boundary rules that keep the swap path from faulting.

On ps2gl, state transitions and texture binds often cost more than vertices. Batch by state first. A tiny quad that reconfigures the full pipeline can be more expensive than dozens of vertices under one already-correct state block.

The text path proved it in stages:

- per-glyph `DrawTexturePro` was about 70 full batch setups for the gameplay HUD;
- batching each string saved several milliseconds;
- batching across strings saved more by avoiding per-string font texture binds;
- batching HUD rects was the real win, because each previous rect reset the full GL/GS pipeline for four vertices.

General rule: hot paths should use `begin/add/end`, hoist state into `begin`, and vary only uniform color or data inside.

Keep one uniform color per `r1Begin / r1End` batch. ps2gl has no reliable unlit varying-color renderer. Mixed colors inside one batch can fault, hang, or route into an unsupported renderer. `r1End` alone does not necessarily flush; adjacent same-mode/same-texture batches can merge. If colors vary per primitive, either flush each primitive or bucket by quantized RGBA and emit one uniform-color batch per bucket.

This applies to 2D and 3D. Speedlines bucket by alpha. Particles bucket by quantized RGBA. Lens flare ghosts have few distinct colors, so one flush per ghost is already fine.

Do not call raylib or timer APIs immediately after `EndDrawing()` on PS2. The raylib4ps2 `EndDrawing()` path flushes and swaps, and the main loop expects GS housekeeping immediately afterward. A `GetTime()` call in that narrow slot faulted. Measure logic and render before `EndDrawing()`, then derive swap from frame time if needed.

Avoid `%f` formatting on PS2's libc in per-frame HUD/debug strings; it has crashed. Use integer fixed-point formatting such as `%d.%02d`.

Chained translucent passes should share one state block. If water splash and engine jets both want lighting off, depth writes off, alpha blend on, `PGL_CLIPPING` off, and edge AA off, the caller should set that state once, call both helpers, and restore once.

Public PS2 draw primitives should fence their own state. Internal add calls between begin/end can assume the begin state, but public entry points must not depend on what the previous pass happened to leave behind.

Chapter 8 - Depth, Blending, and 2D Primitives

Relevant manuals: GS User's Manual Ch 3 (Drawing Function) — the ZTST pixel test, alpha-blending equation and its $0x80 = 1.0$ factor convention, the texture environment (MODULATE), and the rasterization fill rule all live here.

Two GS facts drive this whole chapter and are worth holding in mind: the GS has no separate "depth test enabled" bit (disabled depth simply is `ZTST=ALWAYS`), and its alpha-blend factor treats 128, not 255, as 1.0. Almost every 2D and overlay bug below is one of those two facts leaking through an OpenGL-shaped API that pretends otherwise.

`glDisable(GL_DEPTH_TEST)` is unreliable on ps2gl for overlay passes. ps2gl's cached flag can say depth is already disabled while the GS draw environment still has a real Z test. For depth-independent overlays, force the pass function instead: enable depth test, set `glDepthFunc(GL_ALWAYS)`, disable depth writes, draw, then restore `GL_LEQUAL`. On the GS, "disabled" depth is effectively an always-pass mode anyway.

Untextured alpha-blended primitives do not fade correctly. ps2gl scales untextured vertex alpha by 255, while the GS blend factor treats 128 as 1.0. Upper-half alpha values clamp or over-blend. If something must fade, make it textured with a 1x1 white texture under `GL_MODULATE`.

raylib's 2D shape functions are unsafe or slow on PS2. They often emit untextured per-vertex color, which hits the missing unlit renderer. Use a PS2 shape helper that binds a GS-alpha-scaled 1x1 white texture, uses a single uniform color, disables culling, forces correct alpha blend, and flushes so it does not merge into a neighboring varying-color batch.

Do not use raylib's built-in shapes texture for alpha-blended PS2 fills unless it has been GS-alpha-corrected. A 255-alpha 1x1 white texture saturates the GS alpha scale and turns dim quads fully opaque. Also force 1x1 textures to nearest filtering; hardware linear filtering can sample adjacent VRAM outside a 1x1 texture even when PCSX2 appears fine.

The GS quad diagonal matters. Textured quads split along one diagonal showed a 1-pixel seam on glyphs. The working vertex order for raylib's `DrawTexturePro` is `BL, TL, TR, BR`, which makes ps2gl's quad shader produce the cleaner diagonal. Integer snapping, half-pixel offsets, explicit triangles, and the old OpenGL 0.375 translation did not fix it.

The old desktop OpenGL `glTranslatef(0.375,0.375,0)` hack is unnecessary on PS2. The GS rasterization convention differs; keep the hack removed.

Chapter 9 - Geometry, Clipping, Projection, and Display Lists

Relevant manuals: GS User's Manual Ch 3 (Drawing Function) and Ch 4 (Image Data Transmission) for what reaches the rasterizer; VU User's Manual and VU Instruction Manual for the VU1 transform/clip path ps2gl uses; EE Overview Manual Ch 3 (Functional Overview) for the EE→GS data flow the software clipper participates in.

The PS2 has no hardware near-plane clipper the way a desktop GPU does — geometry that crosses the view plane or runs off the fixed-point coordinate range does not get safely discarded, it wraps into garbage. That single absence is why this chapter exists: every working 3D path here is really an answer to "who clips this, and against what guard band," whether that is the EE software clipper, a proven bounding-sphere bypass, or a whole-triangle cull.

Raw world-space `glBegin` inside `BeginMode3D` does not work reliably on raylib4ps2. raylib's camera matrix and ps2gl's current `GL_MODELVIEW` can be desynchronized. Use one of the proven routes:

- display-list models via rgl transforms and `glCallList`;
- software-clipped meshes through `DrawMeshSH` or `DrawMeshSHNear`;
- screen-space/2D effects after `EndMode3D`;
- special eye-space raw GL for perspective textured quads with identity modelview.

For perspective-correct rotating textured quads, use the `DrawMeshSH` recipe: `BeginMode3D` to set projection, flush rgl, set modelview identity, submit eye-space vertices with raw `glBegin(GL_TRIANGLES)`, and let projection produce the W used for STQ interpolation. 2D screen-space quads interpolate texture coordinates affinely and wrinkle when tilted.

`SHBasis` must track camera roll, not just yaw and pitch. Build it from camera vectors:

```
forward = normalize(target - position);
right = normalize(cross(camera.up, forward));
up = cross(forward, right);
```

This keeps software-clipped geometry aligned with raylib's camera and `GetWorldToScreen`, especially during banked flight.

`PGL_CLIPPING` has two regimes:

- raw display-list models need it on by default, with a bounding-sphere bypass only when the whole model is provably in front of the near plane and inside a tight guard band;
- software-clipper meshes have already been clipped on the EE, so `PGL_CLIPPING` should be off or ps2gl can still reject distant valid draws.

Particles are not an exception. With `PGL_CLIPPING` on, distant explosions thin out. But a particle is one triangle, so do not run the full software clipper on it. Do a cheap whole-triangle near plus four-side guard-band cull, submit raw, and drop any particle crossing a plane. Defer `r1Begin` until the first survivor so an all-culled bucket never emits a zero-vertex GIF packet.

Projective effects must be bounded. Anything using $1/z_c$ can explode as z_c approaches zero. PC and DC viewport clipping can hide that; PS2 GS coordinates wrap into garbage. Clamp projected offsets, fade near singularities, and reject NaN/Inf by bit inspection because `-ffast-math` can delete ordinary non-finite guards.

Display lists capture texture handles at compile time. They do not keep the source `Model` alive, but `UnloadModel` frees the textures the list references. Either keep model textures live for the list lifetime or delete/recompile the display list when loading/unloading the model. In the case study, the logo follows the PC/DC scene lifecycle by deleting the compiled list before unloading the model and recompiling when the model returns.

Display-list model culling is asset-specific. The F22 uses reversed winding and needs explicit culling state. Enemies mirror the player install-once shape but use a simpler uniform tint path instead of per-stage lighting.

Stream per-frame generated geometry through `begin/add/end` helpers. Do not pack several kilobytes of temporary primitive data on the EE stack before calling the backend; the stack can corrupt and surface as an instruction-fetch TLB exception. The water splash tooth emitter is the canonical pattern: `begin` sets state and resets a cached mesh, `add` appends each tooth, `end` clips and draws once.

Flat per-vertex color needs a faked flat light. `ps2gl` has no unlit per-vertex-color renderer — every `PerVtxMaterial:kDiffuse` microprogram also requires `Lighting:1`, and per-vertex color is only transferred to VU1 when `GL_COLOR_MATERIAL` is enabled. With color-material off you silently get the last `glColor` for the whole batch; with it on but lighting off the lookup finds no renderer and faults. To render flat per-face colors (a procedural city of per-color building faces) without real shading, give every vertex the same constant normal and a single directional light pointing along that same eye-space normal, so $N \cdot L \equiv 1$ everywhere and the diffuse term passes the vertex color straight through unshaded. Use `glColorMaterial(GL_FRONT_AND_BACK, GL_DIFFUSE)` (only `GL_DIFFUSE` varies per vertex on `ps2gl`), zero ambient and specular, and configure the light under an identity modelview so its direction matches the emitted normal. The color must also survive the EE clipper, so the clip vertex carries RGBA and interpolates it across cuts.

Chapter 10 - Texture Formats, Alpha, and Filtering

Relevant manuals: GS User's Manual Ch 2 (Local Memory) for the PSMCT32/16/8/4 pixel-storage modes, CLUT storage, and page/block geometry; Ch 3 (Drawing Function) for texture mapping, the TEXA alpha expansion, and filtering; Ch 8 (Details of GS Local Memory) for addressing;

`ps2textures.htm` as a quicker companion to the PSM/CLUT tables; GS User's Manual Supplement for errata.

GS texture work is a VRAM-residency problem first and an image-quality problem second. The GS has 4 MB of local memory carved into fixed-size pages, and a texture is only cheap if it fits and stays resident; the choice of pixel-storage mode (32-bit vs 16-bit vs 8-bit paletted) is mostly a choice about how many pages you spend and therefore what stays resident. This chapter is that economy, plus the alpha and alignment traps that bite once you leave plain RGBA32.

Size textures to on-screen footprint and GS residency, not source resolution. The expensive case is not "large texture" by itself; it is a texture too large to stay resident, causing eviction and re-upload on the draw that needs it. Oversized HUD atlases both cost upload time and evict other textures.

Use console-scale art as the default PS2 source, not HD sources. HD PNGs can decode to many megabytes in EE RAM and fault when an image decoder walks past allocation failure. Keep the original aspect unless intentionally changing art.

PSMCT16 halves VRAM for opaque art. A 512x512 RGBA32 texture is 128 GS pages and does not fit the default 64-page largest slot; 16-bit 5551 is 64 pages and fits. Pack in GS bit order: red in the low five bits, then green, blue, alpha in bit 15. Remember the cost: only one alpha bit. Cutout sprites, fonts, and smooth alpha art should not use this path.

PSMT8 paletted textures are the practical workhorse. They use 8-bit indices plus a 256-entry 32-bit CLUT, giving quarter-size texture bodies with full CLUT alpha. The case study eventually moved the full PS2 texture set to PSMT8 at full console-target resolution.

PSMT8 rules:

- reorder CLUT entries for GS CSM1 layout at build time;
- pack opaque CLUT alpha as `0x80`, not `0xFF`;
- make each texture own its CLUT in the ps2gl fork, because a single global CLUT causes two paletted textures to sample the last uploaded palette;
- reserve kPsm32 GS slots for CLUTs, because `CMMClut` allocates a 16x16 kPsm32 area and cannot see a slot declared as PSMT8;
- all mip levels of one PSMT8 texture share the base texture's one CLUT.

Every DMA-fed embedded blob needs 16-byte alignment. When multiple texture blobs were packed into one generated `.s`, blobs after a 4-byte size field landed at address 4 mod 16. The EE DMAC truncated the source address to a qword boundary and the CLUT upload read four bytes early, shifting the palette by one entry and turning index zero black. Emit `.balign 16` before every blob, not just once per file.

Smooth color ramps are poor palette-alignment tests. Use a sharp grid probe with distinct colors at neighboring entries. If PCSX2 hardware and software agree on wrong colors, suspect data/alignment,

not sampling.

Texture alpha stored in GS memory needs the GS scale. Procedural alpha textures should run through `ps2_image_alpha_to_gs_range()` before upload. Runtime `glColor4ub` is already normalized by ps2gl; do not rescale runtime colors again unless bypassing ps2gl.

Texture filter and wrap state changed over the life of the fork. Current rule: state every asset's filter explicitly. Point-sampled font atlases and 1x1 solid textures should be nearest. Bilinear 1x1 textures can flicker on hardware. When using raw `glTexParameteri`, pass ps2gl's `GL_REPEAT` and `GL_CLAMP` symbols, not desktop numeric constants; ps2gl's enum values differ.

Skybox and floor UV magnitudes matter on real GS hardware. Large absolute UVs cause wobble because the GS perspective-correct interpolation loses precision as S/T magnitude grows. Keep UVs local to slices or wrap the integer tile count out of scrolling UVs. PCSX2 can hide this.

Anamorphic texture storage is a valid VRAM lever: store one axis squeezed and let GS sampling stretch it back. The loader must report logical dimensions, not stored dimensions, or any code deriving UVs, sprite rects, or atlas frame sizes will compensate in the wrong direction.

Chapter 11 - Manual Mipmaps, Dither, and GS Texture Precision

Relevant manuals: GS User's Manual Ch 3 (Drawing Function) for texture LOD selection, mip filtering (MMIN/MMAG), and dithering (DIMX/DTHE); Ch 7 (Registers) for the TEX0/TEX1 fields (MXL, MMIN, K, L, LCM) and the MIPTBP1/2 mip base pointers.

Mipmaps on this stack are a build-it-yourself feature: the GS hardware does them natively through TEX1 and MIPTBP, but ps2gl ships no path to drive those registers, so this chapter is the manual upload recipe. It pairs with dithering because both are about fighting the precision loss of a 16-bit framebuffer — one in texture LOD, one in the final store — and both fail in instructive, hardware-only ways that an emulator renders perfectly.

ps2gl did not ship a mipmap path. Real GS mipmaps require:

- `TEX1.MXL` and `TEX1.MMIN` on the texture's environment;
- explicit `MIPTBP1` and `MIPTBP2` base pointers for levels;
- per-level resident textures;
- locked mip slots so ps2gl's allocator does not reuse them;
- an LOD bias in `TEX1.K`, often strongly negative for grazing floors because the GS has no anisotropic filtering.

MIPTBP addresses need not be contiguous. Upload each mip as its own resident texture and harvest each GS base pointer. Do not rely on automatic contiguous mip addressing.

MIPTBP is global enough to bite. With more than one live mipmapped floor, the new texture can stomp the previous texture's mip pointers. The case-study policy: only one mip texture live at a time, release the old pyramid before creating the new one, and deep-unload floors during stage changes.

The GS supports trilinear (`MMIN = 5`) in one pass. The Dreamcast/PVR trilinear black-surface rule does not apply to GS. But real hardware blends mips with limited precision. PCSX2 can show perfectly smooth trilinear gradients while the real GS shows faint stepped bands. Use a mip-tint probe to verify whether MIPTBP/MXL/MMIN are working and where transitions land.

A 16-bit framebuffer needs GS dithering. The GS has DIMX plus DTHE for this, and ps2stuff's draw environment already has a default matrix. Enable dither through the drawenv object, not a raw register write, because ps2gl re-sends the drawenv and will clobber ad hoc writes. Gate the effective value to 16-bit layouts.

Texture-space dither does not fix magnified framebuffer banding. The sun disc case showed why: bilinear magnification low-passes texel-frequency dither back into a smooth ramp, then the 16-bit framebuffer quantizes it again. Screen-space dither or fewer quantizing additive passes may help, but the baked texture dither experiment was reverted.

Chapter 12 - GS and EE Memory Management

Relevant manuals: EE Core User's Manual for the R5900 caches and memory system (why IOP-DMA'd data needs the uncached mirror); EE User's Manual for the DMAC and scratchpad; GS User's Manual Ch 2 and Ch 8 (Local Memory) for the GS page/slot layout the VRAM allocator manages.

There are two separate memory budgets on a PS2 and they fail differently: EE RAM (32 MB, where decoded assets and the geometry buffers live) overflows into TLB faults, while GS VRAM (4 MB of paged local memory) overflows by silently failing to make a texture resident. This chapter is about measuring both honestly — and the non-obvious facts that textures are double-resident by default, and that for VRAM the number that matters is the largest *single* free slot, not the total free page count.

Audit raylib4ps2's immediate buffer first. `pglInit(128*1024, ...)` allocates about 16 MB across double-buffered immediate geometry arrays. The case study reduced this to `32*1024` qwords and reclaimed about 12 MB of EE RAM while keeping large headroom. Display-list models do not use that immediate buffer; software-clipped raw geometry does.

Use `mallinfo()` for live EE heap readings. On PS2, `uordblks` and `arena` tend to be close because the heap does not return pages to the system, but it is still useful live data.

Textures are double-resident by default: their pixels live in EE RAM as DMA sources and in GS VRAM after upload. This is useful for lazy re-upload after eviction, but it costs memory. If reclaiming EE memory becomes important, be explicit about how an evicted texture can be rebuilt from embedded data.

Add a real GS VRAM query in the fork. `pglPrintGsMemAllocation()` is useful but stdout-only. The valuable live numbers are total pages, used pages, and largest free slot. The largest free slot matters because a texture must fit in one slot. Page count alone does not answer residency.

Default GS slot maps are as important as texture byte size. A 512x512 texture is 32 pages at 8-bit, 64 pages at 16-bit, and 128 pages at 24/32-bit. If the largest free slot is 64 pages, full-res 24/32-bit cannot fit regardless of total free pages.

Chapter 13 - Effects and Scene-Specific Rendering

Relevant manuals: GS User's Manual Ch 3 (Drawing Function) for perspective- correct texturing, alpha blending, and the precision behavior behind UV wobble; VU User's Manual for the transform path. Case-study specifics come from the development notes.

This chapter is the most game-specific in the handbook, but the techniques generalize: how to couple sky/floor/haze geometry so pieces never poke through each other, how to keep projective effects (speedlines, lens flare) from exploding near their $1/z$ singularities, and how to keep UV magnitudes small enough that the GS's reduced-precision perspective division does not visibly swim. Treat the specific effects as worked examples of those three ideas.

Skybox, floor, haze, and water-wall geometry are coupled. Keep them on the same polygon chord endpoints and slice assumptions so haze or floor pieces do not poke past the sky cylinder. The PS2 helper meshes bake absolute floor Y; the world anchor moves them in X/Z only. Do not subtract `world.origin.y` a second time.

The underwater wall's vertical scroll is intentional. It reads as rising current. Keep the under-cap and wall as separate meshes/draws because the cap uses floor forward scroll while the wall adds its own vertical scroll.

Translucent haze ports cleanly through the software clipper. Build CPU-side mesh arrays, use GS-scaled texture alpha, draw after opaque environment, and avoid exact-coordinate z-fighting. On PS2 the haze renderer is its own clipped path; on Dreamcast a different depth-state rule may apply.

Speedlines need PS2-specific projection discipline. Derive the vanishing point from player yaw, manually project with signed `zc`, clamp the drawn VP offset, fade based on horizontal offset near the yaw singularity, and reject non-finite coordinates by bit inspection. The discontinuity at 90 degrees is mathematically real; hide it with fade rather than using `abs(zc)`, which puts the VP on the wrong side.

Speedline alpha is texture-modulated on PS2. Because varying vertex colors are unsafe, bake the tail-to-head ramp into a texture and use uniform alpha per quad/bucket. Match head brightness to the cross-platform path and accept the slightly brighter tail.

Lens flare occlusion should test the sun disc, not only the center. Sample the center plus cardinal perimeter points, ray-test against model OBBs, and scale flare alpha by visible sample fraction. Spheres over-occlude long thin models; world AABBs fail when the model rotates. Inverse-rotated OBB slab tests match the model orientation and avoid rebuilding boxes.

Particles need both sim and renderer policy. Simulation uses priority eviction so explosions beat sparks and sparks beat trails. Rendering uses PGL clipping off, whole-triangle culling, white texturing for alpha fade, quantized RGBA buckets, and a no-empty-batch guard.

A fixed-step inner loop must restore every global it borrows. The attract recorder restored `game.time` but not `game.dt`, causing the cinematic camera to run at frame speed after deterministic sim ticks. On 60 Hz consoles the bug was invisible; on uncapped PC it shifted choreography and made the PS2 renderer look guilty for missing bullets that no longer existed at that timestamp.

Clamp bad `dt` values immediately after reading frame time. Use `if (!(dt > 0.0f) || dt > max)` so NaN and negative values reset too. NaN state is sticky and can poison cameras, positions, and debug displays permanently.

Chapter 14 - Input, Debug UI, and Save Data

Relevant manuals: EE Overview Manual Ch 2 for system context (the IOP owns pad and memory-card services, the EE reaches them over SIF RPC). libpad and libmc behavior is ps2sdk-specific; the rules here are practical, from the case study.

Input, the debug UI, and save data are grouped because they share one theme: they all cross the EE↔IOP boundary through libraries that are unforgiving about buffer placement and init ordering. The recurring failure here is not a wrong value, it is a DMA writing into a stack buffer and corrupting a return address — so the rules are mostly about *where* memory lives (file-scope, aligned, uncached) and *when* you initialize (before raylib touches SIF RPC).

Map libpad into the shared `PadState` early. Cross, Circle, Square, Triangle map to A, B, X, Y in the xbox-style game abstraction; d-pad, sticks, and digital triggers fill the shared fields. Keep raw libpad state for PS2-only debug inputs such as SELECT or shoulders and for the input debug overlay.

Debug menu platform rows should be platform-real. FPS target is `PC_ONLY`, not PS2. PS2 is display/vblank locked. PS2's Graphics submenu owns video mode 448i/448p, aspect 4:3/16:9, anti-aliasing, dithering, screen fit/pos where implemented, and memory-card save target.

GS edge AA on opaque models can read as a black cel outline rather than soft anti-aliasing. Default it off and expose it as a debug option rather than assuming it is a visual improvement.

For memory-card saves, initialize libmc before GL/raylib init. `mcInit()` calls `sceSifInitRpc(0)` unconditionally; running it late can desync ps2link fileio. Load `rom0:SIO2MAN`, `rom0:MCMAN`, and `rom0:MCSERV`, then `mcInit(MC_TYPE_MC)`. Use the plain modules, not the `X*` variants.

libmc result and transfer buffers must be file-scope and 64-byte aligned. The RPC DMA writes into them; stack buffers can corrupt return addresses and surface as instruction-fetch exceptions.

Decode `mcGetInfo` and `mcSync` results carefully. `0` and `-1` can both mean an OK card state for this flow. `-2` is unformatted, and values below about `-10` indicate no card. `mcClose` commits through MCMAN's cache; successful write plus close persisted across power-off in hardware tests.

Under ps2link, libmc RPC bursts can leave the next unrelated fileio call broken. A single blocking stdout write heals the shared ps2link path. This is a dev-loader artifact, not a real hardware save bug, but centralize the barrier so it is understood rather than scattered as mysterious prints.

A BIOS-browsable save needs a directory with `icon.sys` and an `.icn` model. Missing either appears as "Corrupted Data." The icon model is a tiny animated 3D format with 16-bit fixed-point positions/normals/UVs and a 128x128 BGR555 texture. Write it once and avoid rewriting it on every save.

For `.icn` geometry, follow the BIOS camera conventions:

- the BIOS/mymcplus reference effectively negates Y and Z on load;
- the icon rotates/zooms around model point `(0, 2.5, 0)`;
- written Y around `-2.5` centers the model on that pivot;
- keep geometry tiny, such as a textured cube or card;

- wind faces CCW outward;
- flip the texture 180 degrees.

The case study's GLB-to-ICN converter captures those rules.

Chapter 15 - SPU2 SFX Fundamentals

Relevant manuals: SPU2 Overview Manual Ch 1 (Overview of SPU2) for the ADPCM waveform format and 2-core/24-voice layout; Ch 2 (Sound Generation) for voice generation, the PITCH math (§2.1.2), ADSR envelopes, and mixing/VMIX; Ch 3 (Register List) for the VOLL/VOLR/PITCH/ADSR/KON register fields.

The SPU2 is a local-memory ADPCM synthesizer, not a stream player — it does one thing extremely well (per-voice pitch and volume on samples resident in its 2 MB of RAM) and nothing else. This chapter is how to feed it: the IOP-address gotcha in the upload path, the encoder quality gap that makes a naive ADPCM tool sound like AM radio, and the voice-pool and block-flag conventions that separate a looping layer from a one-shot. Streaming long music is hard enough to get its own chapter (Chapter 16).

Use `libsd/ps2snd` direct voice control for game audio that needs continuous runtime pitch and per-voice register control. `audsrv` is not the right abstraction for that job.

Load both `libsd.irx` and `ps2snd.irx`, in that order. `rom0:LIBSD` is not the EE-facing RPC server, and `libsd.irx` alone is not enough. `sceSdInit` binds to the PS2SND RPC registered by `ps2snd.irx`; without it, the bind loop hangs.

`sceSdVoiceTrans` takes an IOP address as source. EE pointers are meaningless to the IOP module. Allocate an IOP buffer, DMA EE data to IOP, then ask `libsd` to DMA IOP data to SPU2. The SPU destination argument is the SPU byte address cast as a pointer value, not the address of a local variable containing that byte address.

Do not chunk through one reused IOP bounce buffer unless the previous IOP to SPU DMA has fully completed. Reusing it early overwrites in-flight audio data and sounds like gritty radio noise. The simple correct path for resident SFX is one IOP buffer sized for the whole sample, one EE-to-IOP DMA, one `sceSdVoiceTrans`, wait with `sceSdVoiceTransStatus`, then free the IOP buffer.

SPU2 pitch is native and should be used. For 48 kHz samples, `PITCH = 0x1000` means unity playback. The usable range is `1..0x3FFF`, roughly -12 to +2 octaves. Encode samples at 48 kHz so unity pitch is exact.

For continuous loops whose amplitude is driven by game mix code, use instant attack and flat full sustain ADSR so `VOLL` / `VOLR` are the actual level. Update pitch and volume per frame as needed.

SPU2 voices are PS-ADPCM. The stock `ps2adpcm` encoder sounded poor because it lacked noise shaping and robust predictor handling. A good encoder should choose best filter/shift per block, noise-shape quantization error, keep predictor history continuous across blocks, and clamp input peaks to avoid overflow.

Loop and one-shot ADPCM block flags differ:

- loops set first block `0x06` and last block `0x03`;
- one-shots set only the final block `0x01` END flag.

One-shots should use a round-robin voice pool. Re-keying a still-playing voice is an acceptable evict-oldest policy. In the case study, loops use voices 0-3, one-shots use 4-11, and music uses 22/23.

Keep shared mix math shared. The case study moved target volumes, smoothing, ducking, pan, and SFX suppression into common code so PC/web, Dreamcast, and PS2 differ only at the leaf "set this voice/stream" layer.

Chapter 16 - Music Streaming and Custom IOP Modules

Relevant manuals: SPU2 Overview Manual Ch 2 (Sound Generation) for the stream input model, the loop/repeat block flags, and VMIX mixing; EE Overview Manual Ch 2/3 for the EE↔IOP split and SIF RPC; EE User's Manual for the DMAC paths the transfers ride on.

Music is where the SPU2's "not a stream player" nature collides with reality: a song is too big to fit in SPU2 RAM, so you must continuously refill the half of a ring buffer the voice is not currently reading. The hard-won conclusion of this chapter is architectural — that refill loop belongs on the IOP as a custom IRX, not on the EE render thread — and getting there means writing your first IOP module and learning why the stock streaming API deadlocks.

Do not use ps2snd's stream API for music in this project shape. `sndStreamOpen` deadlocked the IOP on both `host:` and legacy `mass:`. Its model opens files on the IOP inside blocking RPC/stream machinery, which collides with ps2link fileio and lazy USB mounting.

The working design is a custom SPU2 ring streamer:

- two voices, one per stereo channel;
- each voice uses a two-half ring in SPU2 RAM;
- the file is chunk-interleaved `[L chunk][R chunk]...`;
- block flags make the ring self-loop;
- `NAX` tells which half the hardware is currently reading;
- refill the half the play cursor just left;
- rewind at EOF for looping tracks;
- keep VMIX bits routed into the dry mix.

Run the streamer on the IOP, not the EE. An EE-side streamer worked but caused frame hitches and GS margin artifacts during screen changes because file reads and SPU DMAs touched the render thread. Moving file I/O and SPU DMA to a custom IRX made the EE send only play/stop/volume/pause RPCs and eliminated per-frame audio work.

The IOP module's RPC handler should not do blocking file reads or SPU DMA. It should copy the request, signal a stream thread, and return after consuming the command. The stream thread opens, primes, polls `NAX`, reads, and transfers. Poll with a short `DelayThread`; avoid SPU IRQ file I/O.

Music and SFX share core-wide VMIX registers. Both owners must OR their bits in without clearing the other side. In the case study, the EE SFX VMIX accumulator is seeded with music voices 22/23, while the IOP read-modify-writes to preserve SFX bits.

Custom IRX build traps:

- build with the IOP toolchain and run through `iopfixup`;
- provide your own `irx_imports.h`;
- import `memcpy` / `memset` from `sysclib` when using `-nostdlib -fno-builtin`;
- read files on the IOP through `iomanX_*`, not EE `fileXio`;
- avoid `*/` inside block-comment prose in import headers.

For USB, use the BDM/iomanX stack and the `mass0: device: iomanX.irx`, `usbd.irx`, `usbmass_bd.irx`, `bdm.irx`, and `bdmfs_fatfs.irx`, in order. This is not legacy `usbhdfs` `mass:`. The BDM stack mounts eagerly; a failed open usually means "not mounted yet", so retry.

If the EE reads from iomanX instead, add `fileXio.irx`, call `fileXioInit()`, and use `fileXio*`. Define `NEWLIB_PORT_AWARE` before including `fileXio.h` if needed and do not mix those descriptors with newlib `open()`.

When reading memory the IOP DMA'd into EE RAM, use the uncached mirror `addr | 0x20000000`. The EE data cache may still hold old lines.

Plain burned data CD/DVD media are illegal media on stock or FMCB-only consoles.

`sceCdGetDiskType()` reports illegal media and reads return zeros. `cdrom:` via `cdfs` is useful for authenticated discs, modchip scenarios, or emulator ISOs, not for unmodded network-boot dev. Use USB or HDD for dev removable storage.

Chapter 17 - Validation Doctrine

Relevant manuals: all of them, but this chapter is mostly empirical — it is about how to *use* the manuals and the emulator together rather than any single register.

If the handbook has one meta-lesson, it is this chapter: a PS2 emulator and a PS2 console are different witnesses, and the gap between them is exactly where the expensive bugs live. The discipline below — validate the right things on hardware, prove an object exists before debugging why it "disappeared," compare PCSX2's hardware and software renderers to localize data-vs-sampling bugs — is what turns a multi-day ghost hunt into a one-screenshot diagnosis.

Validate PS2 video modes on hardware. PCSX2 can hide bad `magH`, color/Z page mismatches, overscan position, and display-register mistakes.

Validate texture precision and filtering on hardware. PCSX2 can hide skybox UV wobble, real GS trilinear quantization, and 1x1 linear-filter VRAM fetch artifacts.

Validate standalone boot separately from ps2link boot. ps2link can mask missing IOP reset/SBV setup and can introduce its own memory-card/fileio interop hazard.

When a renderer "loses" an object, first prove the object exists at that instant. The attract bullet investigation burned time in clipping and draw-state probes when the timeline bug meant the bullets were no longer being submitted.

When diagnosing GS margin flashes, distinguish framebuffer contents from BGCOLOR. If the flash appears only outside the active 640x448 frame, fix BGCOLOR timing rather than clears or fullscreen fills.

When diagnosing color/palette corruption, compare PCSX2 hardware and software renderers. If both match the same wrong output, suspect uploaded data, DMA alignment, or CLUT binding.

When optimizing, bisect by pass before inventing abstractions. Text looked like the HUD hog; rect pipeline reconfiguration was the larger cost. The GS and ps2gl are often state-bound, not vertex-bound.

Do not publish speculative lessons as proven facts. Keep a working engineering log if you like, but promote entries into a public manual only after they have survived real validation.

Chapter 18 - ps2gl and ps2stuff Fork Delta

Relevant manuals: GS User's Manual Ch 7 (Registers) for the register fields the fork drives — `PRIM.AA1`, `DTHE / DIMX`, `TEX1`, `MIPTBP1/2`, `DISPLAY` — and Ch 5 (CRTC) for the display state the runtime-mode helpers touch; EE User's Manual for the DMA paths; VU User's Manual for the VU1 context behind ps2gl's clipping and immediate rendering.

The preceding chapters describe scars on the game side; this one describes what had to change *inside* the libraries to make those fixes possible, and — for anyone maintaining their own ps2gl/ps2stuff fork — which of those changes look upstreamable versus project-local. Read it as both a changelog and a design critique: several of the deltas are correct-but-narrow, and the text flags where a cleaner general API should eventually live.

This chapter summarizes the fork-delta report for the case-study ps2gl and ps2stuff forks. The rest of the manual describes the scars from the game side; this chapter describes what changed in the libraries and which parts are likely useful to the broader PS2 SDK ecosystem.

The high-level delta categories are:

- build proof canaries;
- GS memory usage query APIs;
- runtime display mode and raster offset control;
- centered viewport scaling for overscan/screen fit;
- GS edge anti-aliasing exposure;
- GS dither control through the draw environment;
- per-texture CLUT ownership for PSMT8 textures;
- PSMT8 convenience uploaders;
- manual GS mipmap upload for PSMCT16 and PSMT8 textures;
- manual mip texture lifetime and release tracking;
- a Python replacement for the old VU `gasp` preprocessing step.

`ps2stuff` owns the GS memory allocator, so memory reporting starts there. `CMemManager::GetMemInfo()` walks locked slots and every slot list, accumulates total carved pages, counts bound or locked slots as used, and tracks the largest single free slot. The largest free slot is the number to care about. A texture allocation must fit in one compatible slot; total free pages can lie to you.

`ps2gl` exposes that as a C-facing wrapper:

```
extern void pglGetGsMemInfo(int *total, int *used, int *largestFreeSlot);
```

The dependency chain is:

```
application
-> ps2gl: pglGetGsMemInfo()
-> ps2stuff: CMemManager::GetMemInfo()
-> ps2stuff: CMemSlotList::AccumMemInfo()
```

For display positioning, the ps2stuff fork adds a narrow DISPLAY-only register push. The point is to shift the raster without re-sending unrelated display state such as `PMODE`, `DISPFB`, or `BGCOLOR`. The current helper writes `DISPLAY2`, matching ps2gl's active read-circuit assumption. A more general upstream API should choose `DISPLAY1` or `DISPLAY2` explicitly.

The corresponding ps2gl API is:

```
extern void pglSetDisplayOffset(int screen_x, int screen_y);
```

The dependency chain is:

```
application
-> ps2gl: pglSetDisplayOffset()
-> ps2gl: CDisplayContext::SetDisplayOffset()
-> ps2stuff: CDisplayEnv::SetDisplay2()
-> ps2stuff: CDisplayEnv::SendDisplayPos()
-> GS DISPLAY2
```

Runtime video-mode control is exposed through:

```
extern void pglSetVideoMode(int interlaced, int overscan_mode, int screen_x, int screen_y);
```

This updates ps2gl's display interlace state, reprograms FB2, and uses different display magnification for interlaced and progressive output. In the progressive path the fork uses full buffer height and `magH = 2`, fixing the hardware bug where `magH = 4` made DTV output exactly twice too wide while PCSX2 hid the problem. This API does not rebuild GS memory layouts. Applications that change dimensions or pixel formats still need to rebuild draw/display buffers and slot maps themselves.

Centered viewport scale exists as:

```
extern void pglSetViewportScale(float sx, float sy);
```

It persists `vpScaleX/Y`, folds them into the draw context's GS scale, and updates the drawenv scissor to a centered sub-rectangle. This is not a general OpenGL viewport replacement. It is a PS2-friendly screen-fit/squish primitive for overscan and CRT calibration, bounded to $0 < s \leq 1$ in the case-study fork.

GS edge anti-aliasing is exposed as a ps2gl capability:

```
#define PGL_EDGE_AA 3
pglEnable(PGL_EDGE_AA);
pglDisable(PGL_EDGE_AA);
```

The fork tracks the draw-context state and sets `GS::tPrim.aa1` when giftags are generated. This exposes the GS primitive edge AA bit. It does not solve texture shimmer, texture minification, or aliasing inside textured surfaces.

GS dithering is exposed through the drawenv object:

```
extern "C" void pgl_enable_dither(int enable);
```

The important bit is not the function name; it is the route. Dither must live in the `CDrawEnv`, followed by `DrawEnvChanged()`, so `ps2gl` re-sends the correct state. A raw register poke is temporary and will be overwritten by the next `drawenv` send. A cleaner upstream shape would likely be `pglEnableDither()` or a `PGL_DITHER` capability.

Per-texture CLUT ownership is the most direct correctness fix. The original manager-global CLUT model fails with multiple simultaneous PSMT8 textures: the latest `glColorTable()` effectively changes the palette for other indexed textures. The fork gives each `CMMTexture` an owned `CMMClut`, has `SetCurClut()` attach the palette to the currently bound texture, and has `UseCurTexture()` load that texture's own CLUT. This should be reviewed for display-list behavior and deletion edge cases, but the concept is broadly upstream-relevant.

The PSMT8 upload convenience API is:

```
extern "C" unsigned int pgl_create_index8(  
    const void *indices,  
    int w,  
    int h,  
    const void *clut  
);
```

It wraps the correct `GL_COLOR_INDEX` / `GL_UNSIGNED_BYTE` upload plus a 256-entry RGBA CLUT upload. The assumptions must be documented: the CLUT is 16-byte aligned, is already in the storage order expected by this path, and carries GS alpha conventions.

Manual mipmaps are the largest architectural change. PSMCT16 uses:

```
extern "C" unsigned int pgl_create_mip16(  
    void **levels,  
    const int *lw,  
    const int *lh,  
    int count,  
    int kbias,  
    int min_filter  
);
```

PSMT8 uses:

```
extern "C" unsigned int pgl_create_index8_mip(  
    const void **levels,  
    const int *lw,  
    const int *lh,  
    int count,  
    const void *clut,  
    int kbias,  
    int min_filter  
);
```

Both paths create a base GL texture, upload hidden resident mip textures, lock the mip slots, set base `TEX1` fields (`MXL`, `MMIN`, `K`, `LCM`, `L`, `MMAG`), and write `MIPTBP1/2`. The dependency chain is:

```
application texture loader
-> ps2gl: pgl_create_mip16() or pgl_create_index8_mip()
-> ps2gl: CMMTexture::SetMipLevels()
-> ps2gl: pgl_send_miptbp()
-> GS TEX1 + MIPTBP1/2
```

Release uses:

```
extern "C" void pgl_delete_mips(unsigned int baseId);
```

That registry unlocks and deletes the hidden mip textures before the base texture is deleted separately. In the case-study fork the registry is deliberately small and pragmatic. A more upstreamable design should make mip levels owned by the base texture and should re-emit MIPTBP as texture state, otherwise multiple simultaneous mipped textures remain unsafe.

The VU preprocessor replacement is `vu1/gasp.py`, a Python subset of the old `gasp` macro features needed by the current VU sources. It supports includes, macros, conditional assembly forms, repeats, `.equ`, macro parameters, `\@`, and `\&var`. Regeneration is gated behind `REBUILD_VU1=1`; normal builds expect checked-in `.vsm` output. Treat this as a compatibility tool, not a full assembler preprocessor.

Canary prints are project diagnostics, not upstream candidates. Their purpose is to prove the build linked the intended fork rather than toolchain-installed archives. The same idea is useful in any project with nested forks, but the exact print strings should remain project-local.

The fork report's upstreaming read is:

- likely good candidates: memory info query, `pglGetGsMemInfo()`, DISPLAY-only position push, runtime display offset, edge AA exposure, dither exposure, and per-texture CLUT ownership;
- candidates needing design work: runtime video-mode switching, centered viewport scale, PSMT8 upload helpers, and full mipmap support;
- project-local as-is: canary prints, relative sibling include paths, fixed-size mip registry, and raw one-shot MIPTBP DMA as public behavior.

The largest unresolved library-design issue is mipmap state ownership. The fork proves real GS mipmaps work through `TEX1` and `MIPTBP1/2`; the next step is making those registers part of `ps2gl` texture binding instead of assuming one global owner.

Appendix A - Quick Rules

- Standalone boot: reset IOP, wait sync, re-init SIF RPC, apply SBV patches, then load buffer IRX modules.
- ps2link success is not standalone success.
- Use GS color breadcrumbs when `printf` is unavailable.
- Keep the PS2 backend as a backend library. No scene ownership, no direct game state reads, no hardcoded asset filenames.
- Avoid libgraph on top of ps2gl.
- Use 448p, not 480p, for the real progressive layout.
- Align framebuffer height to GS page rows.
- Pair 16-bit color with 16-bit Z; pair 32-bit color with 24-bit Z.
- DTV progressive needs the correct display magnification.
- Re-zero GS BG_COLOR every frame and before long mid-frame stalls.
- Batch by state. State changes beat vertex count as the usual cost center.
- Keep one uniform color per batch, or bucket by quantized color.
- Never emit empty `rlBegin` / `rlEnd` batches.
- Treat the slot after `EndDrawing()` as off-limits for raylib/GS calls.
- Do not use `%f` in hot PS2 HUD/debug formatting.
- For always-on-top overlays, use `glDepthFunc(GL_ALWAYS)`, not only `glDisable(GL_DEPTH_TEST)`.
- Alpha-blended PS2 geometry that must fade should be textured, even if the texture is a 1x1 white pixel.
- Custom world-space geometry goes through the software clipper or a proven display-list path, not raw world-space `glBegin`.
- PGL clipping on for raw display lists unless a bounding-sphere test proves a safe bypass; PGL clipping off for software-clipped meshes.
- Keep UV magnitudes small on GS hardware.
- Use PSMT8 for most art; reserve CLUT slots and align blobs to 16 bytes.
- Only one live mip texture unless the engine explicitly manages MIPTBP ownership.
- Enable 16-bit framebuffer dither through the ps2gl drawenv object.
- Use `mallinfo()` for EE memory and a forked ps2stuff query for live GS memory.
- For GS memory, track largest free slot, not only total free pages.
- Runtime raster offset wants a narrow DISPLAY write, not a full display-state resend.
- If exposing dither through ps2gl, route it through the drawenv object.
- Multiple PSMT8 textures need per-texture CLUT ownership.
- Manual GS mips work, but MIPTBP ownership must be explicit.
- Load both `libsnd.irx` and `ps2snd.irx` for EE-side `sceSd*`.
- `sceSdVoiceTrans` source is an IOP address; SPU destination is the byte address value.
- Encode SPU2 samples at 48 kHz; `PITCH 0x1000` is unity.
- Loops use loop flags; one-shots use END only.
- Music streaming belongs on the IOP in a custom IRX, not the EE render thread.
- `mass0:` is BDM/iomanX, not legacy `mass:`.
- Read IOP-DMA'd EE memory through the uncached mirror.
- Burned data discs are illegal media on stock/FMGB consoles.

Appendix B - Case Study: HyperSolar PS2 Port in One Page

The PS2 target started as a skeleton platform branch: a PS2 source set, a backend translation unit, no-op audio, placeholder rendering, and enough input to cycle states end-to-end.

Milestone 1 brought up the Ninja Dynamics splash using embedded PNG data, GS-alpha-correct texture upload, and the eye-space raw GL perspective-quad path.

Milestones 2 and 3 folded the PS2 PoC environment into shared title rendering: skybox, floor, sun, haze, speedlines, lens flare, logo display list, shared font atlas, and PS2-safe texture/filter handling.

Enemy and player models use display lists with cached AABB data, explicit texture lifetime handling, and PS2-specific clipping/culling state.

HUD performance moved from per-glyph/per-rect reconfiguration toward grouped font and rect batches, proving state changes were the dominant cost.

Texture work evolved from oversized RGBA/16-bit experiments to a full PSMT8 CSV pipeline with per-asset settings, CLUT ownership fixes, 16-byte blob alignment, manual mips for floors, dithered quantization, and stage-resident loading.

Progressive video became a real 640x448p GS layout switch with 16-bit color/Z, drawenv dithering, shared screen dimensions, and a black-bracketed transition.

Audio came up in layers: pitch-clean SPU2 loops, shared mix math, one-shot voice pool, custom ADPCM encoder, then music streaming through a custom IOP IRX and BDM `mass0:` storage.

Saves reached PS2 memory cards with screen settings, aligned libmc buffers, ps2link fileio barrier, BIOS browser icons, and a `glb2icn` converter.

The final rule of the port is simple: the shared game owns meaning, timing, assets, and scene order; the PS2 backend owns the hardware-safe way to draw, store, stream, and persist those decisions.